

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***ASSIGNMENT COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures & Patterns
Assignment number and title: 2 - Iterators
Due date: Sunday, 13 April, 2025, 23:59
Lecturer: Dr. Markus Lumpe

Your name: _____ **Your student ID:** _____

Marker's comments:

Problem	Marks	Obtained
1	44	
2	64	
Total	108	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____

```

1  #include "FibonacciSequence.h"
2
3  // Constructor - Initializes the sequence with by setting
   fPrevious to 0 and fCurrent to 1.
4  FibonacciSequence::FibonacciSequence() noexcept
5      : fPrevious(0), fCurrent(1)
6  {
7  }
8
9  // Operator* - Returns a constant reference to the current
   Fibonacci value.
10 const uint64_t& FibonacciSequence::operator*() const
   noexcept {
11     return fCurrent;
12 }
13
14 // Prefix Increment operator - Advances the Fibonacci
   sequence.
15 FibonacciSequence& FibonacciSequence::operator++()
   noexcept {
16     uint64_t temp = fCurrent;           // Save the
   current value
17     fCurrent = fCurrent + fPrevious;    // Calculate the
   next Fibonacci number
18     fPrevious = temp;                  // Update
   fPrevious to the old current value
19     return *this;                      // Return the
   updated object
20 }
21
22 // Postfix increment operator
23 FibonacciSequence FibonacciSequence::operator++(int)
   noexcept {
24     FibonacciSequence temp = *this;    // Saves the
   current state
25     ++(*this);                          // Advances the
   iterator
26     return temp;                        // Returns the old
   state
27 }
28
29 // Equality operator - Checks if two FibonacciSequence
   objects are equal by comparing their fPrevious and
   fCurrent values.

```

```
30 bool FibonacciSequence::operator==(const FibonacciSequence
    & other) const noexcept {
31     return (fPrevious == other.fPrevious) && (fCurrent ==
        other.fCurrent);
32 }
33
34 // begin - Resets the sequence to its starting values (0
    and 1)
35 void FibonacciSequence::begin() noexcept {
36     fPrevious = 0;
37     fCurrent = 1;
38 }
39
40 // end() - Sets the sequence to a state (fPrevious and
    fCurrent are zero) that does not occur in valid Fibonacci
    generation.
41 void FibonacciSequence::end() noexcept {
42     fPrevious = 0;
43     fCurrent = 0;
44 }
45
```

```

1  #include "FibonacciSequenceIterator.h"
2
3  // Constructor - Initializes the iterator by setting
   fSequence to the provided FibonacciSequence pointer and
   fIndex to the specified starting index.
4  FibonacciSequenceIterator::FibonacciSequenceIterator(
   FibonacciSequence* aSequence, uint64_t aStart) noexcept
5      : fSequence(aSequence), fIndex(aStart)
6  {
7  }
8
9  // Operator* - Returns a constant reference to the current
   Fibonacci number by dereferencing the underlying
   FibonacciSequence.
10 const uint64_t& FibonacciSequenceIterator::operator*()
   const noexcept {
11     return **fSequence;
12 }
13
14 // Prefix Increment operator - Advances the underlying
   FibonacciSequence and increments the iterator index.
15 FibonacciSequenceIterator& FibonacciSequenceIterator::
   operator++() noexcept {
16     ++(*fSequence);           // Advance the
   Fibonacci sequence
17     ++fIndex;                 // Increment
   the index
18     return *this;            // Return the
   updated iterator
19 }
20
21 // Postfix Increment operator - Saves the current iterator
   state, advances the iterator using the prefix operator,
   and returns the saved state.
22 FibonacciSequenceIterator FibonacciSequenceIterator::
   operator++(int) noexcept {
23     FibonacciSequenceIterator temp = *this; // Save
   current iterator state
24     ++(*this);           // Advance to
   the next state
25     return temp;         // Return the
   old state
26 }
27

```

```
28 // Equality operator - Compares the iterator by checking
   // if both fSequence pointers and fIndex values are equal.
29 bool FibonacciSequenceIterator::operator==(const
   FibonacciSequenceIterator& aOther) const noexcept {
30     return (fSequence == aOther.fSequence) && (fIndex ==
   aOther.fIndex);
31 }
32
33 // begin - Resets the underlying FibonacciSequence to its
   // initial state and returns a new iterator with index 0.
34 FibonacciSequenceIterator FibonacciSequenceIterator::begin
   () const noexcept {
35     fSequence->begin();
36     return FibonacciSequenceIterator(fSequence, 0);
37 }
38
39 // end - Returns a new iterator representing the end of
   // the sequence by setting fIndex to MAX_FIBONACCI.
40 FibonacciSequenceIterator FibonacciSequenceIterator::end
   () const noexcept {
41     return FibonacciSequenceIterator(fSequence,
   MAX_FIBONACCI);
42 }
43
```